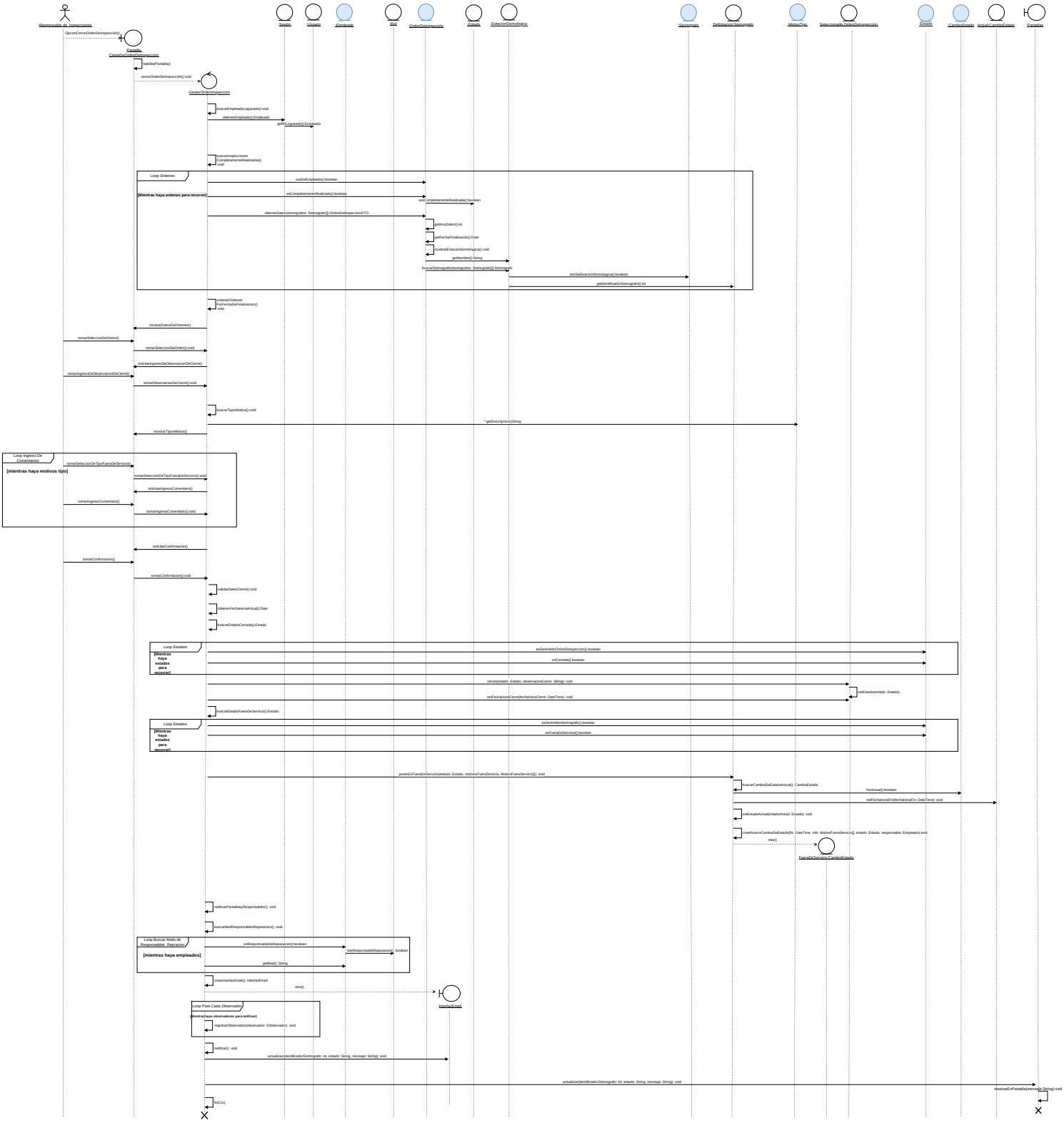
Responsable de Inspecciones
OpcionCerrarOrdenDeInspeccion()
Pantalla CierreDeOrdenDeInspeccion
habilitarPantalla()
cerrarOrdenDeInspeccion():void
GestorOrdenInspeccion
buscarEmpleadoLogueado():void
obtenerEmpleado():Empleado
getEmpleado():Empleado
buscarInspecciones CompletamenteRealizadas():void
Loop Ordenes
[Mientras haya ordenes para recorrer]
esDeEmpleado():boolean
esCompletamenteRealizada():boolean
esCompletamenteRealizada():boolean
obtenerDatosSismografos: SmsmografoID,OrdenDeInspeccionDTO
getNroOrden():int
getFechaFinalización():Date
mostrarEstacionSismologica():void
getNombre():String
buscarSismografo(sismografos: Sismografo[]):Sismografo
esDeEstacionSismologica():boolean
getIdentificadorSismografo():int
ordenarOrdenes PorFechaDeFinalización()
:void
mostrarDatosDeOrdenes()
tomarSeleccionDeOrden()
tomarSeleccionDeOrden():void
solicitarIngresoDeObservacionDeCierre()
tomarIngresoDeObservacionDeCierre()
tomaObservacionDeCierre():void
buscarTiposMotivo():void
* getDescripcion:String
mostrarTiposMotivo()
Loop Ingreso De Comentarios
[mientras haya motivos tipo]
tomarSeleccionDeTipoFueraDeServicio()
tomarSeleccionDeTipoFueraDeServicio():void
solicitarIngresoComentario()
tomarIngresoComentario()
tomarIngresoComentario():void
solicitarConfirmacion()
tomarConfirmacion()
tomarConfirmacion():void
setearDatosCierre():void
obtenerFechaHoraActual():Date
buscarEstadoCerrado():Estado
Loop Estados
[Mientras haya estados para recorrer]
esDeAmbitoOrdenDeInspeccion():boolean
esCerrado():boolean
cerrar(estado: Estado, observacionCierre: String): void
setEstado(estado: Estado)
setFechaHoraCierre(fechaHoraCierre: DateTime): void
buscarEstadoFueraDeServicio():Estado
Loop Estados
[Mientras haya estados para recorrer]
esDeAmbitoSismografo():boolean
esFueraDeServicio():boolean
ponerEnFueraDeServicio(estado: Estado, motivosFueraServicio, MotivosFueraServicio[]): void
buscarCambioDeEstadoActual():CambioEstado
*esActual():boolean
setFechaHoraFin(fechaHoraFin: DateTime): void
setEstadoActual(estadoActual: Estado): void
crearNuevoCambioDeEstado(fh: DateTime, mfs: MotivosFueraServicio[], estado: Estado, responsable: Empleado):void
new()
FueraDeServicioCambioEstado
notificarPantallasyResponsables(): void
buscarMailResponsableReparacion(): void
Loop Buscar Mails de Responsables Reparacion
[mientras haya empleados]
esResponsableDeReparacion():boolean
esResponsableReparacion(): boolean
getMail(): String
crearInterfazEmail(): InterfazEmail
new()
InterfazEmail
Loop Para Cada Observador
[Mientras haya observadores para notificar]
registrarObservador(observador: IObservador): void
notificar(): void
actualizar(identificadorSismografo: int, estado: String, message: String): void
actualizarIdentificadorSismografo: int, estado: String, message: String): void
mostrarEnPantalla(mensaje: String): void
finCU()